Løsningsforslag eksamen DAT107 v2023

Oppgave 1 – JPA/ORM (20% - 48 min)

Vi skal jobbe litt med en database med oversikt over **låt**er og **spillelist**er.

En låt kan være med i **ingen**, **en** eller **flere** spillelister.

En spilleliste inneholder **ingen** (før låter legges til)**, en** eller **flere** låter.

Vi har altså et mange-til-mange forhold. Det er ikke lagret mer data enn koblingene mellom låter og spillelister.

Vi tenker oss følgende tabeller. Det er tatt med eksempeldata for å illustrere mulig innhold.

laat

id (PK) **Surrogatnøkkel**	tittel	artist	varighet
1	'Blue Lights'	'Jorja Smith'	'00:04:10'
2	'Bursdagssang'	'Kaptein Sabeltann'	'00:03:07'
3	'Sober'	'Mahalia'	'00:04:26'
4	'Rosa helikopter'	'Peaches'	'00:03:38'
…	…	…	…

spilleliste

id (PK) **Surrogatnøkkel**	navn	beskrivelse
1	'Laid Back Beats'	'Rolige låter for å komme i stemning'
2	'Maria 10 år'	'Bursdagsmusikk for 10-åringer'
…	…	…

laat_i_liste

laat_id (PK, FK) **Del av sammensatt PK**	liste_id (PK, FK) **Del av sammensatt PK**
1	1
2	2
3	1
4	2
…	…

Vi ønsker å jobbe med denne databasen i et Java-program.

a) (8% ~ 18 min) Skriv Java-klassene for entitetstypene som trengs for å representere eksempelet vist over. Du trenger ikke å skrive metoder, kun instans-/objektvariabler og JPA-annoteringer. *Tips: I et rent mange-til-mange forhold trenger vi kanskje ikke en egen klasse for koblingstabellen.*

   Vi ønsker kun å tilrettelegge for **enveis navigering** i Java mellom låt og spilleliste slik at en spilleliste inneholder de tilhørende låtene, mens en låt **IKKE** inneholder de spillelistene den er med i.

```
Løsningsforslag:

--- MERKNAD! ---------------------------------------------------------------

Det var hintet ganske sterkt om at vi ikke trenger en egen klasse til
koblingstabellen, og en løsning med kun klassene Laat og Spilleliste er både den
enkleste, og den jeg vil gi full score for.

However: Ganske mange besvarelser inneholdt en løsning der koblingstabellen er
representert med en egen klasse (f.eks. LaatIListe), N:M er oppløst i 2x1:N, det
må innføres en sammensatt nøkkel, en egen nøkkelklasse, osv. ... En slik løsning
er mulig, men vil gi redusert score pga. at den er unødvendig komplisert.

Løsningsforslag (begynner på neste side) er vist for begge alternativer.
 - Alt.1: Den ideelle løsningen med kun klassene Laat og Spilleliste
 - Alt.2: Løsning med klasse for koblingstabellen, osv ...

-----------------------------------------------------------------------------
```

Alt.1

```
1   @Entity
    public class Laat {

2      @Id
       @GeneratedValue(strategy = GenerationType.IDENTITY)
       private int id;

       private String tittel;
       private String artist;
1      private LocalTime varighet;

-2     //Trekk hvis denne eller tilsvarende er med!
       @ManyToMany(mappedBy = "laater")
       private List<Spilleliste> medILister;

    }

    @Entity
    public class Spilleliste {

       @Id
       @GeneratedValue(strategy = GenerationType.IDENTITY)
       private int id;

       private String navn;
       private String beskrivelse;

2      @ManyToMany(fetch = FetchType.EAGER)
2      @JoinTable(
            name = "laat_i_liste",
            joinColumns = @JoinColumn(name="liste_id"),
            inverseJoinColumns = @JoinColumn(name="laat_id"))
2      private List<Laat> laater;
}
```

Alt.2

```
4   @Entity
    public class Laat {
       // Som i Alt.1
    }

    // Koblingsklasse
    @Entity
1   @IdClass(LaatIListePK.class)
    public class LaatIListe {

1      @Id @ManyToOne @JoinColumn(name="laat_id")
       private Laat laat;

1      @Id @ManyToOne @JoinColumn(name="liste_id")
       private Spilleliste spilleliste;
    }

    // PK-klasse
1   public class LaatIListePK {
       private int laat;
       private int spilleliste;
    }

    @Entity
    public class Spilleliste {

       @Id
       @GeneratedValue(strategy = GenerationType.IDENTITY)
       private int id;

       private String navn;
       private String beskrivelse;

1      @OneToMany(mappedBy = "spilleliste", cascade = CascadeType.PERSIST)
1      private List<LaatIListe> laaterIListe;
    }
```

Du kan i de videre oppgavene anta at entitetsklassene inneholder de nødvendige konstruktører, gettere og settere, etc. du trenger i løsningene dine.

Vi antar at vi har en hjelpeklasse **LaatDAO**. Du skal lage et par metoder i denne. Du kan anta at en EntityManagerFactory kalt emf er opprettet og tilgjengelig.

b) (3% ~ 8 min) Skriv en metode **`hentSpillelisteMedId(int id)`** i LaatDAO som henter ut en spilleliste med en gitt id. Alle låtene som hører til spillelisten skal også være med i resultatet. Hvis ingen spilleliste med denne id-en finnes, skal det returneres **`null`**.

```
Løsningsforslag:

2   public Spilleliste hentSpillelisteMedId(int id) {

        EntityManager em = emf.createEntityManager();

        try {
1+2+2       return em.find(Spilleliste.class, id);
        } finally {
2           em.close();
        }
    }

    "Alle låtene som hører til spillelisten skal også være med i resultatet.":
1     Den greieste måten å få til dette er å sette
      @ManyToMany(fetch = FetchType.EAGER) ... List<Laat> laater;
      i Spilleliste i a) slik at listen laater hentes automatisk.
```

```
PS! Hvis du bruker try-with-resources (EntityManager er AutoCloseable, og vil da
automatisk lukkes når try-blokken er ferdig) kan løsningen forenkles slik:

2   public Spilleliste hentSpillelisteMedId(int id) {
2       try(EntityManager em = emf.createEntityManager()) {
5           return em.find(Spilleliste.class, id);
        }
    }

1   ... fetch = FetchType.EAGER

Dette gjelder også c) og d).
```

c) (4% ~ 10 min) Skriv en metode **`hentAlleLaaterForArtist(String artist)`** i LaatDAO som henter ut en liste av låter for en gitt artist.

```
Løsningsforslag:

2   public List<Laat> hentAlleLaaterForArtist(String artist) {

        EntityManager em = emf.createEntityManager();

        try {
2            String queryString = "SELECT l FROM Laat l WHERE l.artist = :artist";
2            TypedQuery<Laat> q = em.createQuery(queryString, Laat.class);
2            q.setParameter("artist", artist);

2            return q.getResultList();
        } finally {
            em.close();
        }
    }
```

d) (5% ~ 12 min) Skriv en metode **`opprettSpilleliste(String navn, String beskrivelse, List<Laat> laater)`** i LaatDAO som oppretter og lagrer en ny spilleliste med noen låter (gitt som parameter). Låtene inneholder id. Kun låter som finnes i databasen skal legges inn i spillelisten. (De andre har ugyldig id). Metoden skal returnere spillelisten som er opprettet og lagret i databasen.

```
Løsningsforslag:

    public Spilleliste opprettSpilleliste(
            String navn, String beskrivelse, List<Laat> laater)  {

        EntityManager em = emf.createEntityManager();
        EntityTransaction tx = em.getTransaction();

2       Spilleliste spilleliste = new Spilleliste(navn, beskrivelse);

        try {
            tx.begin();

2           for (Laat laat : laater) {
                if (laatFinnesIDatabase(laat)) { // Hjelpemetode, se nedenfor
                    spilleliste.leggTilLaat(laat);
                }
            }
2           em.persist(spilleliste);
2           tx.commit();

2           return spilleliste;
        } finally {
            em.close();
        }
    }

    private boolean laatFinnesIDatabase(Laat laat) {
        EntityManager em = emf.createEntityManager();
        return em.find(Laat.class, laat.getId()) != null;
    }
```

Oppgave 2 – Modellering (20% - 48 min)

En skole har behov for et datasystem for å håndtere eksamener.

En eksamen gjelder et bestemt fag. Det kan arrangeres eksamen for samme fag et vilkårlig antall ganger på ulike datoer (semester).

Et fag har fagkode, fagnavn og beskrivelse.

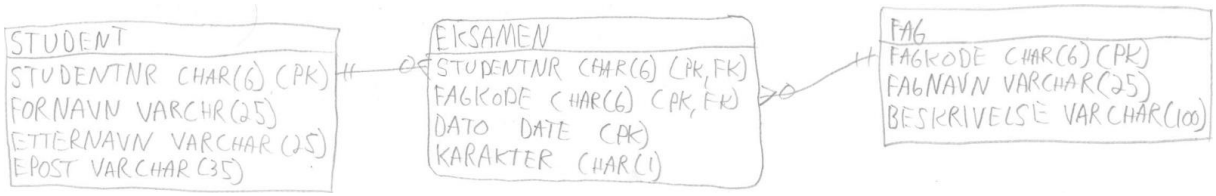
En student har studentnummer, fornavn, etternavn og epost-adresse. En student kan ta eksamen i et vilkårlig antall fag, og kan ta eksamen i samme fag et vilkårlig antall ganger. Hver gang blir det gitt en karakter. Tidligere forsøk på å ta eksamen skal ikke overskrives.

Lag en logisk (fullstendig) ER-modell for denne databasen i en av notasjonene som har vært brukt i kurset.

Besvarelsen skal inneholde:

- Primærnøkler
- Fremmednøkler
- Datatyper
- Min/maks kardinalitet
- Sterke/svake entitetstyper og eksistensavhengighet/uavhengighet (kråkefot-notasjon) eller type aggregering (UML-notasjon)
- **Redegjørelse for de valgene som er tatt. Det blir lagt vekt på både praktisk utførelse og teoretisk forståelse**

Løsningsforslag



Oppgaven kan løses på flere måter, men løsningen ovenfor er trolig den "enkleste" som fullt ut tilfredsstiller kravene.

Student blir identifisert med studentnummer. Dette er en sterk entitetstype siden den ikke arver deler av nøkkel fra en annen tabell. Man kan diskutere hva som er hensiktsmessige datatyper, men for navn og epost er trolig en varchar mest egnet. For studentnummer passer trolig char(6) best hvis man tenker at disse alltid har seks tegn (for eksempel 012345). Ellers er også en int mulig.

Fag blir identifisert med fagkode. Dette er en sterk entitetstype siden den ikke arver deler av nøkkel fra en annen tabell. Man kan diskutere hva som er hensiktsmessige datatyper, men for fagnavn og beskrivelse er trolig en varchar mest egnet. For fagkode passer trolig char(6) best hvis man tenker at disse alltid har seks tegn (for eksempel DAT107).

Eksamen blir identifisert med kombinasjon av studentnummer, fagkode og dato. Dato er nødvendig siden en student kan ta eksamen i samme fag flere ganger, og man skal lagre historikken. Dette er en sterk entitetstype siden den ikke arver deler av nøkkel fra en annen tabell. For studentnummer og fagkode bruker man samme datatype som tidligere. Dato må opplagt være date, mens for karakter passer trolig char(1) best hvis man tenker at disse alltid har et tegn (for eksempel A).

Oppgave 3 – Normalisering (20% - 48 min)

Ta utgangspunkt i følgende unormaliserte (flate) tabell:
Eksamen(fagkode, fagnavn, beskrivelse, studentnummer, fornavn, etternavn, epost, karakter, dato)

Denne tabellen skal normaliseres til 3. normalform.

Besvarelsen skal inneholde:

- Valg av primærnøkkel for opprinnelig tabell (fra eksisterende kolonner uten å introdusere løpenummer/surrogatnøkkel)
- Hva som er (eventuelle) brudd på 1. normalform
- Hva som er (eventuelle) brudd på 2. normalform
- Hva som er (eventuelle) brudd på 3. normalform
- Endelig samling tabeller etter normalisering til 3. normalform (skrevet opp på samme måte som opprinnelig tabell)
- Primær- og fremmed-nøkler for endelig samling tabeller (3. normalform)
- **Redegjørelse for de valgene som er tatt. Det blir lagt vekk på både praktisk utførelse og teoretisk forståelse**

Løsningsforslag

Vi begynner med å bestemme egnet primærnøkkel. Vi ser at tabellen inneholder informasjon om eksamener som en gitt person har tatt i et gitt fag. Primærnøkkel må derfor kunne skille disse ad. Det virker naturlig at studentnummer skal kunne unikt bestemme studenten som har tatt eksamen, mens fagkode skal kunne unik bestemme faget en student har tatt eksamen i. I utgangspunktet kan det da fremstå som at disse to sammen skal kunne være primærnøkkel. Men kan det være aktuelt at en student skal kunne ta eksamen i samme fag flere ganger? Dette virker naturlig, og det er derfor nødvendig å også inkludere dato i primærnøkkel. Dermed får vi:

Eksamen(<u>fagkode</u>, fagnavn, beskrivelse, <u>studentnummer</u>, fornavn, etternavn, epost, karakter, <u>dato</u>)

Vi ser så etter brudd på 1. normalform. Dette er tilfeller hvor vi har "lister" i en kolonne. I utgangspunktet ser det ikke ut som at dette er tilfelle. Muligens kan det være aktuelt for en student å ha flere epost-adresser, men vi har ikke fått opplyst at dette er tilfelle. Vi antar derfor at dette ikke er aktuelt.

Vi ser så etter brudd på 2. normalform. Dette er tilfeller hvor kolonner er avhengige av bare del av primærnøkkel. Vi ser her at fagnavn og beskrivelse bare er avhengig av fagkode, og at fornavn, etternavn og epost bare er avhengig av studentnummer. Dette må vi legge ut i egne tabeller:

Student(<u>studentnummer</u>, fornavn, etternavn, epost)

Fag(<u>fagkode</u>, fagnavn, beskrivelse)

Eksamen(<u>fagkode</u>*, <u>studentnummer</u>*, karakter, <u>dato</u>)

Vi ser så etter brudd på 3. normalform. Dette er tilfeller hvor kolonner er avhengige av andre kolonner som ikke er del av primærnøkkel. Det ser ikke ut som at dette er tilfelle. Tabellene vi har laget blir dermed stående.

Oppgave 4 – SQL (20% - 48 min)

Ta utgangspunkt i følgende tabeller:

Student(studentnummer, navn)
Eksamen(studentnummer*, fagkode, karakter)

Disse tabellene kan minne noe om tabeller i andre oppgaver, men de er ikke nødvendigvis de samme.

Lag SQL-kode for denne databasen.

Besvarelsen skal inneholde:

- Opprettelse av tabellen Eksamen (anta at tabellen Student eksisterer, inkluder fremmednøkler og on update/delete krav)
- Legge inn en ny Eksamen (valgfrie data)
- Slette en Student med studentnummer 42
- Legge til ny kolonne for epost for Student
- Utskrift av alle "gyldige" kombinasjoner av eksamener og studentnavn (en kombinasjon per rad)
- Utskrift av alle "gyldige" kombinasjoner av eksamener og studentnavn som relasjonsalgebra
- Antall eksamener for hver fagkode
- Antall eksamener for hvert student-navn
- Cube-gruppering (group by cube) av eksamener for både fagkode og studentnummer
- Opprette indeks på kolonnen Fagkode i tabellen Eksamen
- **Redegjørelse for de valgene som er tatt. Det blir lagt vekk på både praktisk utførelse og teoretisk forståelse**

Løsningsforslag

Vi oppretter først tabellen eksamen. Studentnummer er en fremmednøkkel til tabellen student (som vi antar allerede eksisterer). Vi bruker datatypen int, men kunne like godt brukt char(6). Vi bruker on update cascade for å angi at endringer i studentnummer skal bli videreført til denne tabellen. Hvor aktuelt det er å endre en students studentnummer kan sikkert diskuteres, så on update restrict for å forby dette hadde heller ikke vært galt. Videre bruker vi on delete restrict for å forby sletting av studenter som har tatt eksamen. Muligens kan man argumentere for andre varianter. Primærnøkkel blir sammensatt av studentnummer og fagkode, siden det er naturlig at en student kan ha mer enn en eksamen, og at en eksamen har mer enn en student. Studentnummer og fagkode får automatisk not null siden disse inngår i primærnøkkel. Muligens burde karakter også ha not null, noe avhengig av om man tenker at en eksamen kan bli lagt inn før karakter har blitt satt. Vi får dermed:

```
Create table eksamen(
studentnr int references student(studentnr) on opdate cascade on delete restrict,
fagkode char(6),
karakter char(1),
primary key (studentnr, fagkode));
```

Vi legger så inn en tilfeldig eksamen:

```
Insert into eksamen (studnr, fagkode, karakter) values (42, 'DAT107', 'A');
```

Så skal vi slette student nummer 42. Da må vi først slette eksamenene denne har, siden vi brukte on delete restrict:

```
Delete from student where studentnr=42;
Delete from eksamen where studentnr=42;
```

Så skal vi legge til en ny kolonne epost for student. Varchar er en naturlig datatype her, og vi bruker ikke not null siden vi ikke vil "kreve" at studenter skal ha epost, særlig hvis det er personlige epost-adresser vi tenker på her:
Alter table student add epost varchar(35);

Neste oppgave er å lage en utskrift av alle eksamener med studentnavn:
Select eksamen.*, student.studentnavn
from student inner join eksamen
on student.studentnr=eksamen.studentnr;

Dette skal så bli gjentatt med relasjonsalgebra. Notasjonen blir ikke vurdert veldig strengt, men det bør være noe lignende dette:
$\pi_{\text{eksamen.*, student.navn}}(\text{student} \bowtie_{\text{student.studentnr=eksamen.studentnr}} \text{eksamen})$

Vi skal så finne antall eksamener for hver fagkode:
Select fagkode, count(*),
from eksamen
group by fagkode;

Deretter skal vi finne antall eksamener for hvert student-navn. Dersom vi også ønsker å få listet ut studenter som ikke har tatt eksamen med null, så må vi bruke en left outer join for å hente ut fullstendig liste av studenter:
select student.navn, count(*)
from student left outer join eksamen
on student.studentnr=eksamen.studentnr
group by student.navn;

Vi skal nå bruke cube-gruppering for både fagkode og studentnummer. Siden vi her ikke trenger student-navn, så kan hele spørringen utføres på tabellen eksamen.
Select studentnr, fagkode, count(*)
from eksamen
group by cube (studentnr, fagkode);

Til slutt skal vi opprette en indeks på kolonnen fagkode i tabellen eksamen. Her kan man innvende at siden fagkode inngår i primærnøkkel, så har databasesystemet "trolig" gjort dette automatisk. Men uansett, vi kan gjøre det manuelt slik:
Create index on eksamen (fagkode);

Oppgave 5 – NoSQL (20% - 48 min)

Ta utgangspunkt Oppgave 4 – SQL.

Implementer denne databasen som en "flat" (data blir gjentatt – ikke normalisert) NoSQL-database. Man kan anta at NoSQL-funksjonalitet i PostgreSQL blir brukt

Besvarelsen skal inneholde:

- Basert på enten JSON-dokument, XML-dokument, nøkkel/verdi-database eller komplekse SQL-datatyper (create type)
- Legge inn en ny Eksamen (valgfrie data)
- Slette en Student med studentnummer 42
- Legge til ny kolonne for epost for Student
- Utskrift av alle "gyldige" kombinasjoner av eksamener og studentnavn (en kombinasjon per rad)
- Antall eksamener for hver fagkode
- Antall eksamener for hvert student-navn
- **Redegjørelse for de valgene som er tatt. Det blir lagt vekk på både praktisk utførelse og teoretisk forståelse**

Løsningsforslag

Oppgaven tillater at ulike typer NoSQL kan blir brukt til å løse den. Vi vil her benytte JSON som et eksempel, men de andre typene vil være like riktige å bruke.

Vi begynner med å opprette en tabell som kan lagre JSON-data. Kolonnen id er et løpenummer som blir brukt til å identifisere JSON-dokumenter, mens kolonnen data inneholder selve JSON-dokumentet.

```
Create table eksamen(
id serial primary key,
data jsonb);
```

Neste skritt er å legge inn en ny eksamen. Her bruker vi en helt flat struktur. Det ville ikke vært galt å bruke en mer kompleks struktur, men det ville gå utover det som er forventet i oppgaven. Vi trenger ikke å oppgi dokumentnummer, siden dette er et serial og blir generert automatisk. Vi får da:

```
Insert into eksamen (data) values ('{"studnr":"42", "navn":"Kari Nordmann", "fagkode":"DAT107", "karakter":"42"}');
```

Nå skal vi slette student nummer 42. Vi må nesten forstå dette slik at alle data relatert til denne studenten skal slettes. Dette kan vi gjøre slik:

```
Delete from eksamen where data->'studennr'='42';
```

Vi skal så legge til en ny kolonne for epost. I en dokumentdatabase så har vi egentlig ikke "kolonner" for ulike attributter. Vi legger bare attributtene til i dataene dersom vi har behov for dette. Her blir det aksepter at man bare legger inn data på nytt med en ny attributt ("kolonne") epost. Det finnes også funksjonalitet for å legge til en ny attributt til et eksisterende dokument, men dette går igjen utover det som blir forventet i oppgaven. Vi får da:

```
Insert into eksamen (data) values ('{"studnr":"42", "navn":"Kari Nordmann", "epost":"kari.nordmann@stud.hvl.no","fagkode":"DAT107", "karakter":"42"}');
```

Neste oppgave er å skrive ut alle eksamener med studentnavn. Dette er veldig enkelt, siden vi allerede har en flat struktur, og det ikke er behov for å kople sammen ulike tabeller. Vi kan for

eksempel gjøre dette slik:

```
Select data->'fagkode', data->'karakter', data->'navn'
from eksamen;
```

Så skal vi finne antall eksamener for hver fagkode:

```
Select data->'fagkode', count(*)
from eksamen
group by data->'fagkode';
```

Og til slutt antall eksamener for hvert student-navn:

```
Select data->'navn', count(*)
from eksamen
group by data->'navn';
```